NS-CausalKT

Neural-Symbolic Causal Knowledge Tracing

A Complete Formula Reference Guide

NS-CausalKT is a hybrid student modelling framework that fuses deep neural sequence learning, symbolic prerequisite reasoning, and causal inference to predict and explain student performance. This document walks through every formula in the system — what it computes, why it exists, and how the pieces combine into a single explainable prediction.

System Components at a Glance

Component	Symbol	Purpose
Neural Knowledge State	h_t	Compressed student history from Transformer + GRU
Concept Mastery Vector	μ_t	Per-concept mastery probabilities (0–1)
Symbolically Corrected Mastery	μ■_t	Mastery adjusted by prerequisite logic
Causal Effect Vector	ACE_vec	Causal importance of each concept
Final Prediction	■_(t+1)	Probability student answers next question correctly

1 Master Prediction Formula

Every component of the model flows into a single linear layer followed by a sigmoid activation. This is the formula that actually outputs the probability a student will answer the next question correctly.

Final Prediction

$$■_{(t+1)} = \sigma(W_p \cdot [h_t ; \mu■_t ; ACE\_vec] + b_p)$$

Concatenates neural state, corrected mastery, and causal scores; passes through a learned linear layer and sigmoid.

Breaking Down Each Symbol

Symbol	Type	Meaning
h_t	Vector	Neural knowledge state at time t (from Transformer + GRU)
μ■_t	Vector	Symbolically corrected concept mastery probabilities
ACE_vec	Vector	Average causal effect of each concept on future success
W_p	Matrix	Learned projection weights (trained via backpropagation)
b_p	Scalar	Bias term
σ(·)	Function	Sigmoid — maps any real number to (0, 1) probability
■_(t+1)	Scalar	Predicted probability of correct answer on next question

Interpretation: ■ = 0.32 means the model estimates a 32% chance the student answers the next question correctly.

2 Input Encoding — e_t

Before anything else, every student interaction at time t — the question asked, whether they answered correctly, which concept was tested, and when — is converted into a single dense vector e_t by concatenating four learned embeddings.

Input Encoding

$$e_t = \text{Embed}(q_t) \oplus \text{Embed}(a_t) \oplus \text{Embed}(c_t) \oplus \Delta t_t$$

⊕ denotes vector concatenation. Each sub-embedding is learned during training.

Sub-embedding	Symbol	What It Captures
Question embedding	Embed(q_t)	Identity and difficulty of the question
Answer embedding	Embed(a_t)	Whether the student answered correctly (0 or 1)
Concept embedding	Embed(c_t)	Knowledge concept being assessed
Time delta embedding	Δt_t	Elapsed time since previous interaction (forgetting signal)

The time delta Δt_t is especially important: longer gaps between interactions let the model represent memory decay without needing an explicit forgetting curve — the GRU learns this pattern from data.

3 Transformer Attention

The sequence of encoded interactions [e_1, …, e_t] is fed through a Transformer encoder. Self-attention lets the model selectively focus on the past interactions that are most relevant to predicting the next

outcome — for example, a student's struggle with fractions three weeks ago may matter more than yesterday's easy drill.

Scaled Dot-Product Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^{\top} / \sqrt{d_k} \right) \cdot V$$

Q = query matrix, K = key matrix, V = value matrix, d_k = key dimensionality.

Transformer Encoder Output

$$H = \text{TransformerEncoder}([e_1, \ldots, e_t])$$

H is the sequence of contextualised hidden states — one per time step.

Term	Meaning
Q (Query)	What the model is currently looking for
K (Key)	What each past interaction offers
V (Value)	The actual content retrieved when a key matches
√d_k	Scaling factor to stabilise gradients for large d_k
softmax(·)	Converts raw scores to attention weights that sum to 1

4 GRU Temporal Refinement — h_t

After the Transformer produces the context-aware hidden states H, a Gated Recurrent Unit (GRU) processes them sequentially to model how student knowledge evolves over time — capturing learning, forgetting, and consolidation.

GRU Update

$$h_t = \text{GRU}(H_t , h_{(t-1)})$$

h_t is the compressed student 'brain-state' vector at time t.

The GRU uses reset and update gates to decide how much of the previous state h_(t-1) to carry forward versus overwrite with new information from H_t. This is what gives the model its memory of earlier struggles even after many subsequent correct answers.

h_t is the first of the three vectors concatenated in the master prediction formula.

5 Concept Mastery Vector — μ_t

A linear projection followed by a sigmoid maps the neural state h_t into a per-concept mastery probability vector. Each element of μ_t represents the model's current estimate of how well the student has mastered one concept.

Mastery Estimation

$$\mu_t = \sigma(W_c \cdot h_t + b_c)$$

W_c and b_c are learned. σ ensures each mastery value lies in (0, 1).

Example output for 5 concepts:

Concept	μ_t value	Interpretation
Fractions	0.80	Strong mastery
Algebra	0.50	Partial mastery
Geometry	0.90	Near-complete mastery
Calculus	0.20	Weak mastery
Probability	0.60	Moderate mastery

6 Symbolic Correction Layer — μ■_t

Raw neural mastery estimates can be logically inconsistent — for instance, predicting high Calculus mastery while Algebra mastery is low, despite Algebra being a prerequisite. The symbolic correction layer enforces domain knowledge encoded as a prerequisite graph G.

6a — Rule Satisfaction Score

For each prerequisite edge (c_j → c_i) in the graph, a satisfaction score measures how well the current mastery state respects that rule:

Rule Satisfaction ϕ(R)

$$\phi(R) = \min(1, 1 - \mu_t[j] + \mu_t[i])$$

If prerequisite j has high mastery but downstream concept i is low, ϕ drops below 1.

6b — Symbolic Gate (Neural-Symbolic Blend)

The final corrected mastery blends the neural estimate with the output of a prerequisite propagation function f_sym over the dependency graph:

Corrected Mastery

$$\mu\blacksquare_t = g \blacksquare \mu_t + (1 - g) \blacksquare f_{sym}(\mu_t, G)$$

■ = element-wise multiplication. g ∈ (0,1) is a learned gate balancing neural vs symbolic.

Symbol	Meaning
g	Learned gate — how much to trust raw neural mastery vs symbolic correction

f_sym(μ_t, G)	Prerequisite propagation: reduces mastery of c_i if c_j (its prereq) is unmastered
G	Concept dependency graph (e.g. Algebra → Calculus)

Example fix: if Algebra mastery = 0.20 but Calculus mastery = 0.90, f_sym pulls Calculus mastery down toward 0.20, enforcing the prerequisite constraint.

7 Causal Layer — Average Causal Effect (ACE)

The causal layer asks a counterfactual question: 'If we were to intervene and ensure the student fully mastered concept c_j, how much would their probability of a correct next answer improve?' This is formalised using the do-calculus operator from Pearl's causal inference framework.

Average Causal Effect

$$ACE(c_j \rightarrow a_{(t+1)}) = E[a_{(t+1)} \mid do(c_j = 1)] - E[a_{(t+1)} \mid do(c_j = 0)]$$

do(c_j = 1) means intervening to set mastery of c_j to 1, regardless of the student's history.

Concrete example for concept 'Fractions':

Scenario	Success Probability	
do(Fractions = 1) — teach fractions	0.85	
do(Fractions = 0) — do not teach fractions	0.60	
ACE = 0.85 – 0.60	0.25	← Strong causal influence

One ACE value is computed per concept, producing the vector ACE_vec. Concepts with high ACE scores are the most impactful intervention targets — this is what makes NS-CausalKT actionable for teachers and learning systems.

8 Loss Functions & Training Objective

The model is trained by minimising a composite loss that jointly optimises prediction accuracy, logical consistency, causal validity, and temporal smoothness.

8a — Prediction Loss (L_pred)

Standard Binary Cross-Entropy between predicted probability ■_t and actual student response a_t ∈ {0, 1}:

L_pred

$$L_{pred} = -\Sigma_t [\, a_t \cdot \log(\blacksquare_t) + (1 - a_t) \cdot \log(1 - \blacksquare_t) \,]$$

Penalises the model for confident wrong predictions.

8b — Symbolic Loss (L_sym)

Penalises the model whenever corrected mastery estimates violate prerequisite relationships. For each edge (c_j → c_i) in the concept graph:

L_sym

$$L_{sym} = -\Sigma_{((c_j \to c_i) \in E)} [\, \mu\blacksquare_t[j] \cdot \log(\mu\blacksquare_t[i]) + (1 - \mu\blacksquare_t[j]) \cdot \log(1 - \mu\blacksquare_t[i]) \,]$$

If prerequisite j is mastered but c_i is not, this loss is large.

8c — Causal Regularisation (L_causal)

Prevents impossible mastery states where a concept is more mastered than all of its prerequisites (with margin γ):

L_causal

$$L_{causal} = \Sigma_k \max(0,\ \mu\blacksquare_t[k] - \max_{(j:(c_j \to c_k))} \mu\blacksquare_t[j] - \gamma)$$

γ is a small tolerance margin. The hinge loss (max 0, …) only fires when violated.

8d — Smoothness Loss (L_smooth)

Prevents unrealistically sudden jumps in mastery between consecutive time steps (knowledge changes gradually):

L_smooth

$$L_{smooth} = \Sigma_t \blacksquare\ \mu\blacksquare_t - \mu\blacksquare_{(t-1)}\ \blacksquare^2_2$$

L2 norm of consecutive mastery differences, summed over the sequence.

8e — Total Training Objective

L_total

$$L_{total} = L_{pred} + \lambda\blacksquare \cdot L_{sym} + \lambda\blacksquare \cdot L_{causal} + \lambda\blacksquare \cdot L_{smooth}$$

λ■, λ■, λ■ are hyperparameters controlling the weight of each auxiliary loss.

Loss	Controls	Why It Matters
L_pred	Prediction accuracy	Core objective — correct answer probability

L_sym	Prerequisite consistency	Ensures mastery respects domain knowledge
L_causal	Causal validity	Prevents mastery exceeding prerequisites
L_smooth	Temporal stability	Knowledge doesn't jump unrealistically

9 End-to-End Example Walkthrough

Suppose a student has a history of struggling with fractions and has never explicitly practised algebra. Here is how NS-CausalKT processes this situation:

Step 1 — Encode: Each past interaction is converted to e_t = Embed(q) ⊕ Embed(a) ⊕ Embed(c) ⊕ Δt.

Step 2 — Attend: Transformer identifies the fractions-related interactions as most relevant via attention.

Step 3 — GRU: h_t captures the cumulative knowledge state: 'weak in fractions, never seen algebra'.

Step 4 — Mastery: μ_t = σ(W_c·h_t + b_c) estimates e.g. Fractions=0.30, Algebra=0.25, Calculus=0.60.

Step 5 — Symbolic fix: Symbolic gate detects Calculus (0.60) > Algebra (0.25) violates Algebra→Calculus. Corrects μ■_t[Calculus] downward.

Step 6 — Causal: ACE_vec computed: Fractions ACE=0.25, Algebra ACE=0.18 → highest-impact intervention targets.

Step 7 — Predict: ■_(t+1) = σ(W_p·[h_t ; μ■_t ; ACE_vec] + b_p) = 0.32 → 32% chance of correct next answer.

Final output: ■ = 0.32. The model also flags 'Fractions' as the highest-ACE intervention — teach this concept first to maximise the student's success probability.

10 Quick Reference — All Formulas

Formula	Expression	Section
Input Encoding	e_t = Embed(q_t) ⊕ Embed(a_t) ⊕ Embed(c_t) ⊕ Δt_t	§2
Attention	softmax(QK■/√d_k)·V	§3
Transformer Output	H = TransformerEncoder([e_1,…,e_t])	§3
GRU Update	h_t = GRU(H_t, h_(t-1))	§4
Mastery Estimation	μ_t = σ(W_c·h_t + b_c)	§5
Rule Satisfaction	ϕ(R) = min(1, 1 – μ_t[j] + μ_t[i])	§6
Corrected Mastery	μ■_t = g■μ_t + (1–g)■f_sym(μ_t,G)	§6
ACE	E[a_(t+1)|do(c_j=1)] – E[a_(t+1)|do(c_j=0)]	§7

Final Prediction	■_(t+1) = σ(W_p·[h_t ; μ■_t ; ACE_vec] + b_p)	§1
Prediction Loss	L_pred = –Σ[a·log(■) + (1–a)·log(1–■)]	§8
Symbolic Loss	L_sym = –Σ_E [μ■[j]·log(μ■[i]) + …]	§8
Causal Regularisation	L_causal = Σ_k max(0, μ■[k] – max_j μ■[j] – γ)	§8
Smoothness Loss	L_smooth = Σ_t ■μ■_t – μ■_(t-1)■²	§8
Total Loss	L_total = L_pred + λ■L_sym + λ■L_causal + λ■L_smooth	§8